Sampled-Ability Algorithm (Conceptual Guide)

Purpose

The sampled-ability algorithm is an optimization strategy for IRT models that tries to reduce bias from treating person abilities as fixed point values while estimating item parameters.

Conceptually, it does this by alternating between:

- **item updates that integrate over person uncertainty**, and
- **person updates that condition on the current item estimates**.

This sits between two extremes:

- fully deterministic joint optimization (fast, but can be brittle in sparse/noisy settings), and
- full Bayesian posterior sampling (more complete uncertainty treatment, but heavier).

High-Level Idea

Let model parameters be split into:

- `theta_person`: person-side parameters (abilities and person-side means/betas),
- `theta_item`: item-side parameters (A/B/C/D and item-side means/betas).

The sampled-ability loop approximates:

1. **Item step:** maximize expected objective over uncertain abilities `E_{q(theta_person)}[log p(data, theta_item, theta_person)]`.
2. **Person step:** maximize objective over person parameters with item values fixed.

Repeated alternation creates a block-coordinate procedure where the item block "sees" uncertainty in person parameters instead of a single hard plug-in value.

Algorithm Structure

1) Parameter Partition

The unconstrained parameter vector is mapped into two disjoint coordinate sets:

- person block indices,
- item block indices.

This mapping is deterministic from model structure and fixed/free parameter masks.

2) Initialization

A person-side optimization step is run first. This avoids early item updates being conditioned on uninformative default person values.

3) Outer Loop (Alternating Updates)

Each outer iteration has two conceptual phases.

3a) Build a Local Person-Uncertainty Approximation

Using the current fit, build a per-person local Gaussian approximation for ability:

- mean at the current ability estimate,
- covariance from local curvature/standard-error information,
- stabilized to remain numerically well-conditioned.

This defines `q(theta_person)` for the current iteration.

3b) Item Update Under Sampled Ability Templates

Convert each person Gaussian into a small deterministic support of ability samples (sigma-point style templates) with associated weights.

Then optimize **only item coordinates**, but evaluate the objective as a weighted average across these templates. In effect:

- person coordinates are treated as fixed per template,
- item coordinates are free,
- objective and gradient are averaged over templates.

This approximates integration over person uncertainty during the item step.

3c) Person Update

With item coordinates fixed at their latest values, optimize **only person coordinates** on the standard objective.

3d) Refresh Summaries

After person update, recompute high-level person distribution summaries (mean/SD/correlation) and optional diagnostics for monitoring optimization behavior.

Stopping Logic (Conceptual)

A scalar convergence signal is built from the two block gradient magnitudes:

- item-block gradient norm,
- person-block gradient norm,
- combined into one joint measure.

The outer loop stops when this combined signal indicates the alternating scheme is no longer making meaningful progress.

The exact numerical threshold is a tuning choice, but conceptually the criterion is: **stop when both block updates are jointly small**.

Why This Helps

Compared with pure point-estimate alternation:

- **Item estimates become less overconfident** in sparse designs because person uncertainty is acknowledged.
- **Optimization can be more stable** when person/item parameters are tightly coupled.
- **Computation remains practical** because uncertainty integration is approximated with a small deterministic support, not full sampling each step.

Mental Model

Think of sampled-ability as:

1. "Given current item parameters, estimate where people probably are (with uncertainty)."
2. "Given that uncertainty cloud of people, update items."
3. "Given updated items, re-estimate people."
4. Repeat until joint progress is small.

It is an uncertainty-aware block optimizer, not a full posterior sampler.

Implementation Design Principles

Good implementations of this algorithm should keep:

- **Separation of concerns:** objective setup separate from per-step optimizer calls.
- **Reusable setup:** expensive target construction reused across outer iterations.
- **Clear block interfaces:** person/item free-index masks explicit and deterministic.
- **Numerical robustness:** covariance stabilization and safe fallbacks for local approximations.
- **Transparent diagnostics:** per-step summaries that explain failures or non-convergence.